

Synopsis — LLM-guided LLVM IR Mutation for Differential Compiler Testing

Project: Can LLMs generate valid and useful LLVM IR tests for differential compiler testing?

Abstract

- This project explores whether large language models (LLMs) can generate or mutate LLVM IR test cases that are both syntactically valid and semantically useful for differential testing. We implemented an end-to-end prototype that generates mutants (LLM, grammar, random), validates them with LLVM tools, and compares LLM-based mutation against grammar/random mutation strategies.

Objectives

- Survey existing compiler-fuzzing and differential-testing approaches (Csmith, YARPGen, coverage-guided fuzzers).
- Catalog structural constraints for valid LLVM IR (SSA, types, PHI correctness, dominance, poison/undef semantics).
- Build a prototype workflow to prompt an LLM for IR mutation and reliably filter outputs.
- Evaluate LLM vs grammar/random mutation on validity, novelty, and usefulness.

Approach (high level)

- Mutator types: `llm` , `grammar` , `random` (implemented in backend). See backend flow in `BACKEND_WORKFLOW.md`.
- Generation pipeline: seed → mutator → deduplicate → write `raw_mutants.json` → validate via `llvm-as` and `opt -passes=verify` → move to `valid_mutants/` or `invalid_mutants/` .
- Comparison: collect logs, build manifest, compute metrics (validity, mismatch, trivial/invalid taxonomy), and inspect failure cases.

Key results (summary)

- Prototype built and integrated with frontend + backend pipeline. Phase 3 artifacts and metrics recorded in the documentation index.
- Real-world validation snapshot: the repo records `111 mutants` validated in Phase 3 and a `100%` test pass rate for the implemented unit tests (see `DOCUMENTATION_INDEX.md` metrics).

Primary conclusion

- LLMs can produce novel IR patterns that may expose different optimizer behaviors, but they also tend to generate a larger fraction of invalid or trivially-useful IR compared with grammar-based mutation. Effective filtering (verification + semantic checks) and seed diversity are required to convert LLM outputs into high-value differential tests.

Artifacts

- Full structured report: [docs/FINAL_REPORT.md](#)
- Graphical knowledge map: [graphify-out/graph.html](#)

Prepared for: Compiler Design coursework (10 marks)